

2024年上半年软件设计师上午真题及解析(总结)

1.在计算机网络OSI协议五层体系结构中, (B)工作在数据链路层。

A. 路由器 B.以太网交换机 C.防火墙 D.集线器

网络层: 路由器、防火墙
数据链路层: 交换机、网桥
物理层: 中继器、集线器

2.软件交付之后, 由于软硬件环境发生变化而对软件进行修改的行为属于(B)维护。

A.改善性 B.适应性 C.预防性 D.改正性

软件维护:

- 适应性维护: 指使应用软件适应信息技术变化和管理需求变化而进行的修改。企业的外部市场环境和管理需求的不断变化也使得各级管理人员不断提出新的信息需求
- 改正性维护: 指为了识别和纠正软件错误、改正软件性能上的缺陷、排除实施中的错误, 应当进行的诊断和改正错误的过程。
- 预防性维护: 系统维护工作不应总是被动地等待用户提出要求后才进行, 应进行主动的预防性维护, 通过预防性维护为未来的修改与调整奠定更好的基础。
- 完善性维护: 扩充功能和改善性能而进行的修改。对已有的软件系统增加一些在系统分析和设计阶段中没有规定的功能与性能特征。

3.以下不属于函数依赖的Armstrong公理系统的是(C)。

A.自反规则 B.传递规则 C.合并规则 D.增广规律

考查数据库基础知识:

函数依赖的 Armstrong 公理及其引理。设关系模式 $R(U,F)$, 其中 U 为属性集, F 是 U 上的一组函数依赖, 那么有以下推理规则。

- 公理-自反律: 若 $Y \subseteq X \subseteq U$, 则 $X \rightarrow Y$ 被 F 逻辑蕴含。
- 公理-增广律: 若 $X \rightarrow Y$ 为 F 所蕴涵, 且 $Z \subseteq U$, 则 $XZ \rightarrow YZ$ 被 F 逻辑蕴含。
- 公理-传递律: 若 $X \rightarrow Y$, $Y \rightarrow Z$ 为 F 所蕴涵, 则 $X \rightarrow Z$ 被 F 逻辑蕴含。

根据上述3条推理规则又可推出下述3条推理规则:

- 引理-合并律: 若 $X \rightarrow Y$ 且 $X \rightarrow Z$, 则 $X \rightarrow YZ$ 为 F 所蕴涵。 (证明: 根据增广律可以得到 $X \rightarrow XY$, $XY \rightarrow YZ$, 再根据传递律得到, $X \rightarrow YZ$ 。)
- 引理-伪传递律: 若 $X \rightarrow Y$ 且 $WY \rightarrow Z$, 则 $XW \rightarrow Z$ 为 F 所蕴涵。 (证明: 证明方法依然是 增广律 和 传递律。)
- 引理-分解律: 若 $X \rightarrow Y$ 且 $Z \subseteq Y$, 则 $X \rightarrow Z$ 为 F 所蕴涵。 (证明: 根据自反律可以得到 $Y \rightarrow Z$, 再根据传递律, 得证 $X \rightarrow Z$ 。)

4.结构化分析方法的基本思想是(B),

A.自底向上逐步分解 B.自顶向下逐步分解 C.自底向上逐步抽象 D.自顶向下逐步抽象

考查软件工程开发方法:

结构化开发方法主要特征包含: 自顶向下、逐步分解求精、严格区分阶段、阶段产生标准化。

5.执行以下Python语句之后, 列表y为 (B)。

```
x=[1,2,3]
y=x+[4,5,6]
A.出错      B.[1,2,3,4,5,6]      C.[5,7,9]      D.[1,2,3,[4,5,6]]
```

在python语言中，两数组相加，+ 属于拼接。

6.对于一棵树，每个结点的孩子结点个数称为结点的度，结点度数的最大值称为树的度。某树T的度为4，其中有5个度为4的结点，8个度为3的结点，6个度为2的结点，10个度为1的结点，则T中的叶子结点个数为(A)。

A.38 B.29 C.66 D.57

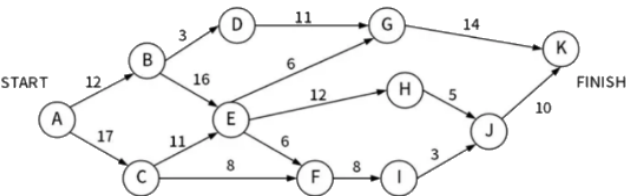
总结点数 = (度数1 * 该度数结点 + ... + 度数n * 该度数结点) + 1 = 边总数 + 1

 = 5 * 4 + 8 * 3 + 6 * 2 + 10 * 1 + 1 = 67

叶子结点 = 总结点数 - 所有度数的结点数

 = 67 - (5 + 8 + 6 + 10) = 38

7.8.下图是一个软件项目的活动图，其中顶点表示项目里程碑，连接顶点的边表示包含的活动，则一共有(B)条关键路径，关键路径长度为(B)。



- A.2 B.4 C.3 D.1
- A.48 B.55 C.30 D.46

考察项目管理关键路径。

如上图可以发现，关键路径为最长的一条路线。关键路径有4条，分别为：ABEHJK、ABEFIJK、ACEHJK、ACEFIJK，总长度为55。

9.对于定点纯小数的数据编码，下述说法正确的是(D)。

A.仅原码能表示-1 B.仅反码能表示-1 C.原码和反码均能表示-1 D.仅补码能表示-1

考察计算机基础码制相关内容。

机器字长为 n 时各种码制表示的带符号数的范围

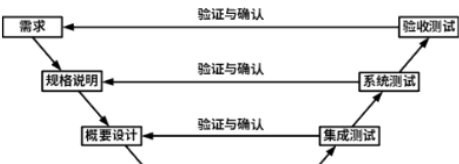
码 制	定 点 整 数	定 点 小 数
原码	$-(2^{n-1}-1) \sim +(2^{n-1}-1)$	$-(1-2^{-(n-1)}) \sim +(1-2^{-(n-1)})$
反码	$-(2^{n-1}-1) \sim +(2^{n-1}-1)$	$-(1-2^{-(n-1)}) \sim +(1-2^{-(n-1)})$
补码	$-2^{n-1} \sim +(2^{n-1}-1)$	$-1 \sim +(1-2^{-(n-1)})$
移码	$-2^{n-1} \sim +(2^{n-1}-1)$	$-1 \sim +(1-2^{-(n-1)})$

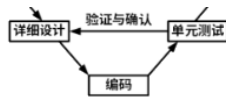
10.软件测试过程中的系统测试主要是为了发现 (D) 阶段的问题。

A.软件实现 B.概要设计 C.详细设计 D.需求分析

考察软件测试内容。

系统测试和验收测试都是针对于需求分析阶段进行测试的。





系统测试是针对整个产品系统进行的测试，目的是验证系统是否满足了需求规格的定义，找出与需求规格不符或与之矛盾的地方，从而提出更加完善的方案。

11.WWW服务器与客户机之间主要采用（ B ）安全协议进行网页的发送和接收。

A.HTTP B.HTTPS C.HTML D.SMTP

12.瀑布模型的主要特点是（ C ）。

A.用户容易参与到开发活动中 B.易于处理可变需求
C.缺乏灵活性 D.用户与开发者沟通容易

瀑布模型的缺点：

- (1) 各个阶段之间产生大量的文档，极大地增加了工作量。
- (2) 由于开发模型是线性的，用户只有等到整个过程的末期才能见到开发成果，从而增加了开发风险。
- (3) 不适应用户需求的变化，并且在需求分析阶段不可能完全获取。
- (4) 在软件开发前期未发现的错误传到后面的开发活动中时，可能会扩散，进而可能会导致整个软件项目自开发失败。

所以，瀑布模型适用于需求明确或很少变更的项目

13.TCP序号单位是（ B ）。

A.赫兹 B.字节 C.比特 D.报文

序列号（SequenceNumber 简称seq）：序列号是TCP报文中用于标识每个字节的唯一数字。它表示在一个TCP连接中发送的字节的顺序。

赫兹是频率的单位；比特是信号量的基本单位；报文是TCP/IP网络中传输的信息单位，而TCP序列号是基于每个数据段中的字节的，而不是整个报文。

14. 采用简单选择排序算法对序列（49，38，65，97，76，13，27，49）进行非降序排序，两趟后的序列为（ A ）。

A, (13, 27, 65, 97, 76, 49, 38, 49)
B, (38, 49, 65, 76, 13, 27, 49, 97)
C, (13, 38, 65, 97, 76, 49, 27, 49)
D, (38, 49, 65, 13, 27, 49, 76, 97)

直接选择排序：从题目可知采用非降序排序，在所有记录中选出最小的记录，把它与第1个记录交换，然后在剩余的记录内选出最小的记录与第2个交换.....依次类推。

例：关键字序列T=（21，25，49，27，16，08）

第1趟 08, 25, 49, 27, 16, 21

第2趟 08, 16, 49, 27, 25, 21

第3趟 08, 16, 21, 27, 25, 49

第4趟 08, 16, 21, 25, 27, 49

第5趟 08, 16, 21, 25, 27, 49

15.在计算机系统 中，CPU中跟踪后继指令地址的寄存器是（ C ）。

A.指令寄存器 B.状态条件寄存器 C.程序计数器 D.主存地址寄存器

- 指令寄存器（IR）：存放即将执行的指令。
- 状态条件寄存器：存放状态标志和控制标志。
- 程序计数器（PC）：存放下一条要执行指令的地址。
- 主存地址寄存器：保存当前CPU访问内存单元的地址。

16.硬盘所属的存储类别是（ D ）。

A.寄存器 B.缓存 C.主存 D.辅存

CPU对应的存储类别：寄存器；

Cache对应的存储类别：缓存；

主存对应的存储类别：DRAM；

辅存对应的存储类别：硬盘、光盘等。

17.UML类图在软件建模时，给出软件系统的一种静态设计视图，用（ C ）关系可明确表示两类事物之间存在的特殊一般关系。

A.聚合 B.依赖 C.泛化 D.实现

考察UML关系：

- 依赖关系：一个事物发生变化影响另一个事物。
- 实现关系：接口与类之间的关系。
- 泛化关系：特殊/一般关系。
- 聚合关系：整体与部分生命周期不同。属于关联关系。

18.在29个元素构成的查找表中查找任意一个元素时，可保证最多与表中5个元素进行比较即可确定查找结果，则采用的查找表及查找方法是（ C ）。

A.二叉排序树上的查找 B.顺序表上的顺序查找
C.有序顺序表上的二分查找 D.散列表上的哈希查找

- 二叉排序树上的查找：在二叉排序树上查找一个元素时，平均查找长度通常与树的深度有关。
- 顺序表上的顺序查找：从头到尾或从尾到头遍历整个列表。
- 有序顺序表上的二分查找：每次比较都会排除一半的元素
- 散列表上的哈希查找：哈希查找的性能主要取决于哈希函数的设计及哈希表的填充因子。

19.算术表达式 $b*(a+c)-d$ 的后缀式是（ D ）。（+、-、*表示算术的加、减、乘运算，运算符的优先级和结合性遵循惯例。）

A.ba+cd*- B.bacd+*- C.ba*c+d*- D.bac+*d-

20.面向对象软件从不同层次进行测试。（ D ）层测试类中定义的每个方法，相当于传统软件中的单元测试。

A.模板 B.系统 C.类 D.算法

一般来说，对面向对象软件的测试可分为下列4个层次进行。

（1）算法层：测试类中定义的每个方法，基本上相当于传统软件测试中的单元测试。

（2）类层：测试封装在同一个类中的所有方法与属性之间的相互作用。在面向对象软件中类是基本模块，因此可以认为这是面向对象测试中所特有的模块测试。

（3）模板层：测试一组协同工作的类之间的相互作用，大体上相当于传统软件测试中的集成测试，但是也有面向对象软件的特点（例如，对象之间通过发送消息相互作用）。

（4）系统层：把各个子系统组装成完整的面向对象软件系统，在组装过程中同时进行测试。

21.循环余校验码（CRC）利用生成多项式进行编码。设数据位为n位，校验位为k位，则CRC码的格式为（ C ）。

A.k个校验位按照指定间隔位与n个数据位混淆 B.k个校验位之后跟n个数据位
C.n个数据位之后跟k个校验位 D.k个校验位等间隔地放入n个数据位中

考察校验码基础知识：

- 奇偶校验码编码方法：由若干位有效信息（如一个字节），再加上一个二进制位（校验位）组成校验码。这个校验位可以加在最前面也可以是最后面。
- CRC的编码方法是：在k位信息位之后拼接r位校验位。

- 海明校验码编码方法：在有效信息位中加入几个校验位形成海明码，使码距比较均匀地拉大，并把海明码的每个二进制位分配到几个奇偶校验组中。

22. 以下关于通过解释器运行程序的叙述中，错误的是（ C ）。

- A. 可以由解释器直接分析并执行高级语言源程序代码
- B. 与直接运行编译后的机器码相比，通过解释器运行程序的速度更慢
- C. 解释器运行程序比运行编译和链接方式产生的机器代码效率更高
- D. 可以先将高级语言程序转换为字节码，再由解释器运行字节码

1. 编译程序和解释程序

高级语言或汇编语言编写的程序称为源程序，源程序不能直接在计算机上执行。

- 如果源程序是汇编语言编写的，则需要一个称为汇编程序的翻译程序将其翻译成目标程序，然后才能执行。
- 如果源程序是为高级语言时，这个翻译程序称为编译程序
- 按源程序中语句的执行顺序，逐条翻译并立即执行相关功能的处理程序、称为解释程序。

2. 解释执行：源程序的每个语句一经解释就立即执行。

- 优点：可移植性较好、开发速度较快、与用户通信方便。
- 缺点：效率低。

23. 进行面向对象系统设计时，若存在包A依赖于包B，包B依赖于包C，包C依赖于包A，则此设计违反了（ D ）原则

- A. 稳定抽象 B. 稳定依赖 C. 依赖倒置 D. 无环依赖

面向对象设计原则：

- 稳定抽象原则：强调的是包的抽象程度与其稳定程度一致。
- 稳定依赖原则：要求包之间的依赖关系都应该是稳定方向依赖的，即包要依赖的包要比自己更具有稳定性。
- 依赖倒置原则：强调的是程序应该依赖于抽象接口，而不是具体的实现，从而降低客户与实现模块间的耦合。
- 无环依赖：强调的是在组件的依赖关系图中不允许存在环。

24. 下列算法属于Hash算法的是（ A ）。

- A. SHA B. DES C. IDEA D. RSA

消息摘要算法，也被称为哈希(Hash)算法或散列算法：MD5、SHA；

对称加密算法：DES、IDEA、AES、RC-5；

非对称加密算法：RSA；

25. 在关系表中选出若干属性列组成新的关系表，可以使用（ A ）操作实现。

- A. 投影 B. 笛卡儿积 C. 选择 D. 差

考查数据库基础SQL语言：

- 投影：投影出某属性列。
- 笛卡尔积：两表之间的乘积，组成新的表之后，新表的属性列为两表之和，元组数为两表之乘积。
- 选择：选择某条件下的一条/多条元组记录。
- 差：两表之间的差集是指在该表中减去两者之间重复的元组。

26. 在撰写学术论文时，通常需要引用某些文献资料。以下叙述中，（ A ）是不正确的。

- A. 既可引用发表的作品，也可引用未发表的作品
- B. 不必征得原作者的同意，不需要向他文付报酬
- C. 只能限于介绍、评论作品

D.只要不构成自己作品的主要部分，可适当引用资料

27.进行面向对象设计时，以下（ B ）不能作为继承的类型。

A.多重继承 B.分布式继承 C.单重继承 D.层次继承

考查面向对象基础：

对于面向对象的设计，支持多重继承和单重继承（多个父类和一个父类），同时也支持层次继承（一般会同时实现继承类实现接口）。

28.在采用定点二进制的运算器中，减法运算一般是通过（ A ）来实现的。

A.补码运算的二进制加法器 B.原码运算的二进制加法器
C.补码运算的二进制减法器 D.原码运算的二进制减法器

考查计算机基础：

在运算中，CPU分为控制器和运算器，执行算术逻辑运算的是算术逻辑运算单元，然后将其结果放到加法器执行。在运算过程中，补码是正确的适合加减运算的，因为+0和-0对应的补码值是一样的。

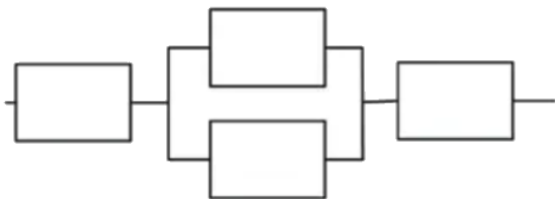
29.用于收回SQL访问控制权限的操作是（ C ）。

A.GRANT B.DELETE C.REVOKE D.DROP

考查数据库基础SQL语言部分：

- GRANT：表示授权
- DELETE：表示删除表内部数据
- REVOKE：表示销权
- DROP：表示删除表结构

30.某系统由下图所示的余部件构成。若每个部件的千小时可靠度都为R，则该系统的千小时可靠度为（ D ）。



A. $(1-(1-R)^2)(1-R)$ B. $(1-R)^2 (1-R^2)$ C. $R(1-R^2)R$ D. $R(1-(1-R)^2)R$

计算机可靠度计算：（每个部件可靠度为R）

- 两个部件串联： R^2
- 两个部件并联： $1-(1-R)^2$

31.已知二维数组A按行优先方式存储，每个元素占用2个存储单元，第一个元素A[0][0]的地址为100，元素A[3][3]的存储地址是220，则元素A[5][5]的地址是（ A ）。

A.300 B.310 C.306 D.296

存储位置 = 起始地址 + $(i*n + j) * \text{size}$

其中i指行号，j指列号，n指每行元素数（数组列数）

由题目可知： $A[3][3] = 220 = 100 + (3n+3)*2$ ，可得 $n=19$

$A[5][5]$ 的地址 = $100+(5*19+5)*2 = 300$

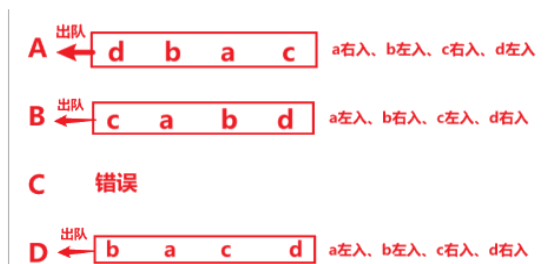
32.关于链表操作中，说法正确的是（ C ）。

- A.新增一个头结点需要遍历链表 B.新增一个尾结点需要遍历链表
C.删除最后一个节点需要遍历链表 D.删除第一个节点需要遍历链表

- A. 新增一个头结点通常不需要遍历链表。只需要修改头结点的指针指向新节点，并让新节点指向原来的头结点即可。
 - B. 如果链表没有设计尾部指针（即只有头部指针），那么新增一个尾结点确实需要从头开始遍历链表直到找到最后一个节点。但如果链表有尾部指针，则可以直接通过尾部指针添加新节点，无需遍历。
 - C. 要删除最后一个节点，通常情况下需要先找到这个节点的前一个节点，然后更新它的指向，使其指向最后一个节点的下一个节点（通常是NULL）。因此，在没有尾部指针的情况下，这通常需要遍历整个链表来找到倒数第二个节点。
 - D. 删除第一个节点不需要遍历链表。只需要更新头结点的指针，让它指向当前头结点所指向的下一个节点，并释放原来的头结点即可。
- 选项C在一般情况下是正确的。请注意，这里的答案取决于链表的具体实现方式。例如，如果链表有一个对尾节点的引用，则添加或删除尾节点将不需要遍历链表。

33.某队列允许在其两端进行入队操作，但仅允许在一端进行出队操作。若元素a、b、c、d依次全部入队列，之后进行出队列操作，则不能得到的出队序列是(C)。

- A.dbac B.cabd C.acdb D.bacd



34.森林的叶子节点是(C)。

- A.二叉树中没有左孩子的节点 B.二叉树中没有右孩子的节点
C.森林中度为0的节点 D.二叉树中度为1的节点

“度”指的是一个节点拥有的子节点数量。叶子节点是指度为0的节点，也就是该节点没有任何子节点。

35.在29个元素构成的查找表中查找任意一个元素时，可保证最多与表中5个元素进行比较即可确定查找结果，则采用的查找表及查找方法是(C)。

- A.二叉排序树上的查找 B.顺序表上的顺序查找
C.有序顺序表上的二分查找 D.散列表上的哈希查找

A. 二叉排序树上的查找：最坏情况下，如果树退化成链表（一边倒的树），那么查找的时间复杂度将是 $O(n)$ ，即需要比较的次数最多可以达到树的高度，也就是29次。但是，如果是一棵平衡的二叉排序树（如AVL树或红黑树），高度大约是 $\log_2(n)$ ，在这种情况下，29个元素的高度大约是5（因为 $2^5 = 32$ ，接近29）。这意味着在最佳情况下，我们可以通过至多5次比较找到元素。所以这个选项是有可能的。

B. 顺序表上的顺序查找：最坏的情况下，需要遍历整个表才能确定目标元素是否存在，这意味着可能需要最多29次比较。显然这不是我们想要的答案。

C. 有序顺序表上的二分查找：二分查找每次将查找区间减半，其时间复杂度也是 $O(\log n)$ 。对于29个元素的表，使用二分查找至多需要5次比较（ $\log_2(29) \approx 4.86$ ），所以这也符合题意。

D. 散列表上的哈希查找：理想情况下，哈希查找的时间复杂度可以接近 $O(1)$ ，但是在最坏的情况下，如果出现哈希冲突，也可能需要多次比较来解决冲突。但是题目要求是最多5次比较，而哈希查找不保证这一点。

考虑到平衡的二叉排序树也可以满足这一条件，理论上A也是一个合理的答案。但在通常的教学或考试环境中，二分查找在有序表上实现的查找效率和稳定性更常被强调，因此最优选的答案可能是C。

36.37.正规集 $(ab|c)(1|2|3)$ ，可以识别的字符种类有(D)个，下列(A)可被匹配。

- A.1 B.2 C.3 D.6
A.ab2 B.abc C.a2 D.123

正规集可分为两部分理解：

- $(ab|c)$ 这一部分意味着可以匹配 "ab" 或者 "c"。
 - $(1|2|3)$ 这一部分意味着可以匹配 "1"、"2" 或者 "3"。
- 组合成的字符种类有6个，分别是：ab1、ab2、ab3、c1、c2、c3。

38.对下列有向图的邻接矩阵，进行深度遍历的次序是(C)。

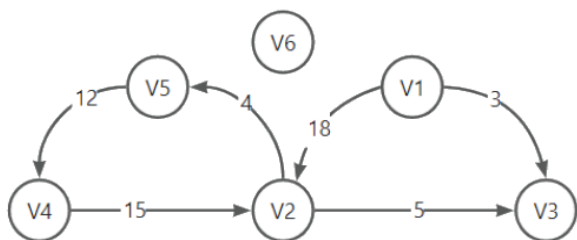
V1	V2	V3	V4	V5	V6
∞	18	3	∞	∞	∞
∞	∞	5	∞	4	∞
∞	∞	∞	∞	∞	∞
∞	15	∞	∞	∞	∞
∞	∞	∞	12	∞	∞
∞	∞	∞	∞	∞	∞

- A.V1-V2-V3-V4-V5-V6 B.V1-V4-V2-V3-V5-V6
C.V1-V2-V3-V5-V4-V6 D.V1-V2-V5-V4-V3-V6

补充上面的有向图的邻接矩阵：

	V1	V2	V3	V4	V5	V6
V1	∞	18	3	∞	∞	∞
V2	∞	∞	5	∞	4	∞
V3	∞	∞	∞	∞	∞	∞
V4	∞	15	∞	∞	∞	∞
V5	∞	∞	∞	12	∞	∞
V6	∞	∞	∞	∞	∞	∞

将邻接矩阵转为有向图：



深度遍历优先方法：首先访问出发顶点V，然后依次从顶点V出发搜索V的任意一个邻接点。它类似树的前序遍历（根左右）。从V1开始进行深度优先遍历，我们可能的路径为：

1. 开始于V1。
2. 从V1出发，可以去往V2或V3。我们先访问V2（根据字母顺序或任意选择）。
3. 在V2处，可以去往V3或V5。我们先访问V3。
4. 在V3处，发现没有其他可以直接前往的节点，于是返回V2。
5. 在V2处，接下来访问V5。
6. 在V5处，可以去往V4。
7. 在V4处，可以去往V2，但由于V2已经被访问过了，因此跳过。
8. V6没有指针指向它，因此如果要访问V6，需要确保在访问完其他节点之后单独处理V6。

题目选择C。其实D的答案也可能对，只不过题目要求只能选择一个答案，需考虑题目所暗示的标准遍历顺序。由于深度优先遍历与权值无关，那估计是用权值顺序来访问相邻点。如果优先选择大的权值，V1->V2，V2->V3，那么C正确，D错误。

39.当一棵非空二叉树的(A)时，对该二叉树进行中序遍历和后序遍历所得的序列相同。

- A.每个非叶子结点都只有左子树 B.每个非叶子结点都只有右子树

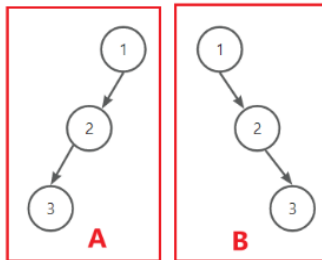
C.每个非叶子结点的度都为1

D.每个非叶子结点的度都为2

优先排查C、D答案，无法确定顺序。如果有三个节点（左节点、右节点、根节点），按前中后序遍历如下：

- 前序遍历：根左右
- 中序遍历：左根右
- 后序遍历：左右根

根据A、B选项画图如下：



A选项：

- 中序遍历：321
- 后序遍历：321

B选项：

- 中序遍历：123
- 后序遍历：321

答案选择A。

40.在CPU调度中，(C)不可能的。

A.放权等待 B.资源等待 C.无限等待 D.定时等待

良好的操作系统设计应该避免无限等待的情况发生。无限等待意味着一个进程或线程永远不能获得所需的资源或条件来继续执行，这会导致系统死锁或性能严重下降。因此，在正常情况下，“无限等待”是不应该发生的，是应该避免的情况。

41.在微型计算机中，管理键盘最适合采用的 I/O控制方式是(D)方式。

A.DMA B.无条件传送 C.程序查询 D.中断

1. **DMA (Direct Memory Access, 直接存储器存取)**：DMA主要用于高速设备的大批量数据传输，如磁盘控制器到内存的数据传输。键盘输入通常是字符级别的，频率不高，不适合使用DMA。
2. **无条件传送**：这种方法适用于那些简单且不需要精确同步的设备。键盘输入需要精确同步，并且需要检测按键的状态变化，因此不适合使用无条件传送。
3. **程序查询 (Programmed I/O, PIO)**：程序查询方式需要处理器不断轮询设备状态寄存器以确定设备是否准备好数据。这种方式会占用大量的处理器时间，特别是在用户频繁按键的情况下，这将严重影响处理器的效率。
4. **中断 (Interrupt)**：中断方式允许键盘在按键按下或抬起时通知处理器，从而使得处理器可以在执行其它任务的同时处理键盘输入。这种方式既节省了处理器的时间，又保证了对键盘输入的及时响应。
由于键盘是一个交互设备，用户期望其输入能够被快速响应，同时不希望这种输入占用过多的处理器资源。因此，采用中断方式管理键盘是最合适的方案。当键盘上有按键动作时，它可以发送一个信号给处理器，告知处理器有输入需要处理。处理器可以处理完当前的任务后，再去响应这个中断请求。

42.FAT文件系统用的(B)文件管理结构。

A.索引 B.基于文件的簇状链式结构 C.链式结构 D.顺序结构

FAT文件系统 (File Allocation Table, 文件分配表) 使用的是基于文件的**簇状链式结构** (Cluster Chain) 来管理文件。

在FAT文件系统中，磁盘被划分为若干个固定大小的块，这些块被称为簇 (Cluster)。文件不是存储在一个连续的区域，而是可以分散存储在多个簇中。每个文件都有一个簇链，指向其在磁盘上的存储位置。这种结构使得文件可以灵活地分布在磁盘的不同位置，提高了磁盘的利用率。

分散存储在整个磁盘上。每个文件由一系列连续的簇组成，这些簇通过一个链表连接起来，形成一个链式结构。FAT表记录了每个簇的状态以及指向下一个簇的指针，从而形成了一个簇链，用来追踪文件的实际存储位置。

选项A（索引）通常指的是索引节点（inode）文件系统，如Unix中的文件系统使用的机制；选项C（链式结构）虽然部分正确，但不够具体，因为FAT文件系统特指使用簇的链式结构；选项D（顺序结构）通常指的是文件存储在一个连续的物理区域。

43.某文件管理系统在磁盘上建立了位示图(bitmap)，记录磁盘的使用情况。若计算机系统的字长为128位，磁盘的容量为1024 GB，物理块的大小为8MB，那么该位示图的大小为(B)个字

A.4096 B.1024 C.2048 D.4098

在位示图中，大小由空间磁盘决定，每一位对应一个物理块，一个字对应（字长）个物理块。即每个字有128个物理块。

该位示图的大小计算（注意单位换算）： $1024 \times 1024 / (8 \times 128) = 1024$

44.下面(D)范式包含多值依赖。

A.1NF B.2NF C.3NF D.4NF

在数据库规范化的过程中，不同的范式（Normal Form, NF）旨在减少数据冗余和提高数据完整性。这四种范式的区别：

• **1NF (第一范式):**

- **目的:** 消除重复组。
- **要求:** 确保表中的每一列都是原子的，即不可再分的最小单位。
- **解决的问题:** 重复字段组，如地址、电话号码等。

• **2NF (第二范式):**

- **前提:** 已经满足1NF。
- **目的:** 进一步减少数据冗余，确保非主键列完全依赖于整个主键。
- **要求:** 在没有复合主键的情况下，所有的非主键列都必须完全依赖于单一的主键，而不是主键的一部分。
- **解决的问题:** 部分依赖导致的数据冗余。

• **3NF (第三范式):**

- **前提:** 已经满足2NF。
- **目的:** 消除非主键列之间的传递依赖。
- **要求:** 非主键列不能依赖于其他非主键列，而只能直接依赖于主键。
- **解决的问题**:** 由于传递依赖导致的数据冗余和更新异常。

• **4NF (第四范式):**

- **前提:** 已经满足3NF。
- **目的:** 消除非平凡的多值依赖。
- **要求:** 除了候选键之外，不存在其他依赖于整个候选键的多值集合。
- **解决的问题:** 多值依赖，即一个非主键列的值集依赖于另一个非主键列的值集，而不是直接依赖于主键。

45.数据库的基本表、存储文件和视图的结构分别对应(D)。

A.用户视图、内部视图和概念视图 B.用户视图、概念视图和内部视图
C.概念视图、用户视图和内部视图 D.概念视图、内部视图和用户视图

三级模式体系结构主要包括外模式（用户模式）、模式（逻辑模式或概念模式）和内模式（存储模式）。

- **外模式:** 用户模式，用户可以看到的数据视图，视图通常是由基本表通过查询语句创建出来的，它为用户提供了一个简化和定制的数据视图。
- **模式:** 数据库的整体逻辑视图，也就是基本表的集合。基本表是用户定义的数据存储单元，它们定义了数据的逻辑结构，包括表名、列名、数据类型等信息。
- **内模式:** 数据库的物理存储视图，也就是数据库是如何实际存储在物理介质上的。存储文件描述了数据在物理存储设备上的组织方式，如存储位置、索引、数据分页等细节。

46.由于知识技术可以同时被多个人使用，所以知识专利具有(D)。

A.双重性 B.独占性 C.地域性 D.实践性

专利的主要特性包括：

- **独占性**：专利给予发明者对其发明的独占使用权。
- **地域性**：专利权只在其被授予的国家或地区内有效。
- **期限性**：专利权是有期限的，通常为20年。

这里强调知识技术被多个人同时使用，强调的是知识技术，而非专利的特性。选择D，该知识技术可以在实践中被广泛应用和共享。

47.程序员甲将其编写完成的软件程序发给同事乙并进行讨论，之后由于甲对该程序极不满意，因此甲决定放弃该程序，后来乙将该程序稍加修改并署自己名在某技术论坛发布。下列说法中，正确的是(D)。

A.乙对该程序进行了修改，因此乙享有该程序的软件著作权
B.乙的行为没有侵犯甲的软件著作权，因为甲已放弃程序
C.乙的行为未侵权，因其发布的场合是以交流学习为自的的技术论坛
D.乙的行为侵犯了甲对该程序享有的软件著作权

- **选项A**不正确，因为即便乙对程序进行了修改，但著作权仍归原作者甲所有。
- **选项B**不正确，因为即使甲放弃了程序，但没有明确声明放弃著作权，乙仍然不能擅自发布修改后的版本。
- **选项C**不正确，发布场合不影响著作权的归属和保护。不论是在何种场合发布，未经许可使用他人作品都是侵权行为。
- **选项D**正确，乙的行为侵犯了甲对该程序享有的软件著作权，因为未经甲的许可，乙无权修改并发布该程序。

48.数据流图中的组成不包括(D)。

A.数据流 B.外部实体 C.数据加工 D.控制流

49.结构化设计方法中使用结构图来描述构成软件系统的模块以及这些模块之间的调用关系。结构图的基本成分不包括(C)。

A.模块 B.调用 C.数据 D.控制

在结构化设计（Structured Design, SD）方法中，结构图（Structure Chart, SC）是用来描述软件系统的高层抽象模型，它展示了系统由哪些模块组成以及这些模块之间的调用关系。结构图的基本成分主要包括以下几个方面：

- **模块 (Module)**：表示系统中的一个功能单元，可以是子程序、函数、过程等。
- **调用 (Call)**：表示一个模块调用另一个模块的关系。
- **控制 (Control)**：表示模块之间的控制流，即控制信息如何在模块间传递。

结构图通常不直接表示数据流或数据存储。结构图主要关注模块及其相互之间的调用关系和控制流，而不是数据流动。数据流动通常在数据流图（DFD）中表示。

50.观察者（Observer）模式适用于(D)。

A.访问一个聚合对象的内容，而无需暴露他的内部表示。
B.减少多个对象或类之间的通信复杂性。
C.将对象的状态恢复到先前的状态
D.一个多对象依赖关系。当一个对象修改后，依赖他的对象都自动得到通知

- **A. 访问一个聚合对象的内容，而无需暴露他的内部表示**：这是迭代器模式或外观模式的目的。
- **B. 减少多个对象或类之间的通信复杂性**：这是职责链模式或中介者模式的目的。
- **C. 将对象的状态恢复到先前的状态**：这是备忘录模式的目的。
- **D. 一个多对象依赖关系。当一个对象修改后，依赖他的对象都自动得到通知**：这是观察者模式的核心定义。

51.在 C/C++ 程序中，对于函数中定义的非静态局部变量，其存储空间在(A)分配。

A.栈区 B.静态数据区 C.文本区 D.自由堆区

- **栈区 (Stack) :** 存放函数的参数值、局部变量等。栈区的内存由编译器自动分配与释放。当函数调用时，函数内的局部变量和参数都会被压入栈中；当函数调用结束时，这些局部变量和参数也会被自动释放。
- **静态数据区 (Static Data Area) :** 存放全局变量和静态变量，以及常量（如字符串字面量）。静态变量在整个程序运行期间都存在。
- **文本区 (Text Area) :** 存放程序的机器码指令，即代码段。
- **自由堆区 (Heap) :** 动态分配的内存区域，通常通过函数如 `malloc()`、`calloc()`、`realloc()` 和 `free()` 来管理。堆区的内存由程序员手动分配和释放。

52.关于 Python，下列说法正确的是(B)。

A.用 try 捕获异常，有 except，无需执行 finally
B.可以使用 raise 关键字来手动抛出异常
C.except Exception 可以捕获所有异常
D.可以用 switch...case 语句表示选择结构

在Python中，异常处理是通过try、except、else、finally等关键字来实现的。

- finally块是可选的，但它通常用于清理工作，无论是否发生异常都需要执行的操作
- raise关键字可以用来抛出自定义的异常或现有的异常类型。
- Exception是大多数异常的基类，使用except Exception可以捕获大部分内置异常，但并非所有异常。例如，SystemExit、KeyboardInterrupt等不会被捕获。此外，如果有自定义异常继承自其他基类而不是Exception，也不会被捕获。
- Python没有内置的switch...case语句。虽然可以通过字典和其他构造来模拟switch...case的功能，但标准语法中没有这样的语句。

53.下列测试方法中，(C)程度最高。

A.语句覆盖 B.判定覆盖 C.路径覆盖 D.条件覆盖

在软件测试中，覆盖率是一种衡量测试充分性的指标，它描述了测试执行过程中源代码被验证的程度。几种常见的覆盖标准及其覆盖程度的比较：

1. 语句覆盖 (Statement Coverage) :

- 要求至少执行一次程序中的每个可执行语句。
- 这是最基础的覆盖标准，仅确保每个语句至少被执行了一次。

1. 判定覆盖 (Decision Coverage 或 Branch Coverage) :

- 要求至少执行一次每个判定语句（如if语句、循环条件）的每个分支。
- 相比语句覆盖，判定覆盖更为严格，因为它不仅关注语句的执行，还关注判定条件的不同结果。

1. 条件覆盖 (Condition Coverage) :

1. 要求至少执行一次每个布尔表达式中的每个条件的真/假值。
2. 条件覆盖比判定覆盖更细致，它关注的是条件表达式中每个单独条件的结果。

2. 路径覆盖 (Path Coverage) :

1. 要求至少执行一次程序中的每条可能路径。
2. 这是最严格的覆盖标准，因为它要求覆盖程序中所有可能的执行路径。

54.55.以太网交换机属于网络模型中(C)，管理的是(B)。

A 网络层 R 传输层 C 数据链路层 D 物理层

以太网帧
A.IP 地址 B.mac 地址 C.报文 D.帧

以太网交换机主要在OSI七层模型中的第二层，即数据链路层工作。- 以太网交换机通过学习数据帧的源MAC地址来构建其内部的转发表。当数据帧到达交换机端口时，交换机会记录该帧的源MAC地址与对应的端口关联。当需要转发帧时，交换机会查找目的MAC地址，并将帧发送到相应的端口，或者在未知目的MAC地址的情况下广播到所有端口（除了接收帧的那个端口）。

56.VLAN 不能隔绝(D)。

A.广播域 B.内网互访 C.内外网 D.攻击和漏洞利用

VLAN（虚拟局域网）的主要作用之一是将网络划分为多个逻辑上的广播域，从而控制广播流量并增强安全性。

- **A. 广播域**：VLAN的主要功能之一就是将网络分割成不同的广播域，因此它可以有效地隔绝广播域。
 - **B. 内网互访**：VLAN可以限制同一VLAN内部主机之间的互访，但默认情况下，如果不配置额外的安全措施，同一VLAN内的主机是可以互相通信的。
 - **C. 内外网**：VLAN本身并不能隔绝内外网之间的通信。通常，内外网的隔离需要通过路由器、防火墙等设备来实现。
 - **D. 攻击和漏洞利用**：VLAN可以提高网络安全，通过隔离不同的用户群体或服务，减少潜在的攻击面。但是，VLAN并不能完全防止攻击和漏洞利用。高级的攻击手段仍然可以通过各种途径绕过VLAN的隔离。
- VLAN不能完全隔绝的是 **D. 攻击和漏洞利用**。答案选D。

57.下面(C)命令可以查看网络配置。

A.ping B.netstat C.ipconfig D.telnet

- **ping**：用于测试网络连通性。可以间接验证网络配置是否正确。
- **netstat**：用于显示网络连接、路由表、网络接口统计等信息。可以显示当前开放的端口和服务。
- **ipconfig**：用于显示所有网络接口的TCP/IP配置信息，包括IP地址、子网掩码、默认网关、DNS服务器等。这是查看网络配置的主要命令。
- **telnet**：用于远程登录到另一台主机上。它不是一个用来查看本地网络配置的命令。

58.下列措施中，(D)可以保证数据的可靠性。

A.访问控制 B.数据加密 C.鉴别 D.异地数据备份

- **访问控制**：访问控制是用来确保只有授权用户才能访问数据的技术，提高数据安全性。
- **数据加密**：数据加密是将数据转换为密文的过程，只有拥有解密密钥的人才能读取数据。提高数据再传输过程中的保密性，用来防止数据泄露。
- **鉴别**：鉴别是用来验证用户身份的过程，提高系统安全性。
- **异地数据备份**：指在不同的地理位置存储数据的副本。在本地数据因灾难性事件丢失时，确保数据可以从备份中恢复。这是直接保证数据可靠性的措施之一。

59.黑客获取整个用户资料信息属于(C)。

A.撞库 B.社工库 C.拖库 D.洗库

- **撞库 (Credential Stuffing)**：指的是黑客使用在其他地方获取的用户名和密码组合尝试登录其他网站或应用。
- **社工库 (Social Engineering Database)**：通常指的是收集了大量个人信息的数据库，这些信息可以通过社会工程学手段获取，用于实施诈骗或其他恶意活动。
- **拖库 (Dump)**：指的是黑客成功攻破了一个网站或应用的数据库，并将整个数据库的数据下载下来。
- **洗库 (Database Cleaning)**：这个术语在网络安全领域并不常用，有时指的是对一些清理或整理数据库的行为。但在上下文中，如果“洗库”指的是对获取的数据进行清洗和整理，以便更好地利用这些数据，那么它也可能与拖库后的数据处理相关。

60.下列算法属于 Hash 算法的是(A)。

A.SHA B.DES C.IDEA D.RSA

- **SHA (Secure Hash Algorithm):** 是一系列加密哈希函数, 包括 SHA-1、SHA-2 和 SHA-3。这些算法将任意长度的输入数据转换为固定长度的哈希值, 常用于数据完整性校验和数字签名。
- **DES (Data Encryption Standard):** 是一种对称加密算法, 用于加密数据。它不是哈希算法。
- **IDEA (International Data Encryption Algorithm):** 也是一种对称加密算法, 用于加密数据。它不是哈希算法。
- **RSA (Rivest–Shamir–Adleman):** 是一种非对称加密算法, 用于加密和数字签名。它不是哈希算法。

61.利用报文摘要算法生成报文摘要的目的是(A)。

A.防止发送的报文被篡改 B.对传输数据进行加密, 防止数据被窃听
C.验证通信对方的身份, 防止假冒 D.防止发送方否认发送过的数据

报文摘要算法 (如 MD5、SHA-1、SHA-256 等) 是一种单向哈希函数, 它可以将任意长度的输入数据转换为固定长度的哈希值 (报文摘要)。

选项:

- **A.防止发送的报文被篡改:** 发送方生成报文的哈希值 (报文摘要), 并将报文和哈希值一起发送给接收方。
- **对传输数据进行加密, 防止数据被窃听:** - 加密算法 (如 AES、DES 等) 的功能。
- **验证通信对方的身份, 防止假冒:** 通常通过数字签名和公钥基础设施 (PKI) 来实现。
- **防止发送方否认发送过的数据:** 通常通过数字签名来实现。数字签名结合了哈希算法和非对称加密算法, 可以确保数据的完整性和发送方的身份。

62.若浮点数用补码表示, 则判断运算结果为规格化数的方法是(C)。

A.阶符与数符相同为规格化数
B.阶符与数符相异为规格化数
C.数符与尾数小数点后第一位数字相异为规格化数
D.数符与尾数小数点后第一位数字相同为规格化数

规格化数的定义: 对于二进制浮点数, 规格化数的尾数部分 (也称为小数部分或 mantissa) 通常要求其最高有效位 (即小数点后的第一位) 为 1。这是因为规格化数的形式可以最大化表示范围和精度。

判断方法: 数符与尾数小数点后第一位数字相异为规格化数。 具体来说, 如果数符 (即符号位) 为 0 (表示正数), 则尾数小数点后的第一位应为 1; 如果数符为 1 (表示负数), 则尾数小数点后的第一位应为 0。。

63.某个程序所有者拥有所有权限, 组成员有读取和运行的权限, 其他用户只有运行的权限, 该程序的权限为(C)。

A.742 B.741 C.751 D.752

文件权限可以用三位八进制数字表示, 每一位数字代表所有者 (user)、组 (group) 和其他用户 (others) 的权限。每个权限位可以是以下组合之一:

- 读取 (r) : 4
- 写入 (w) : 2
- 执行 (x) : 1

所有者: 拥有所有权限 = $4+2+1 = 7$

组成员: 拥有读取和执行权限 = $4+1 = 5$

其他用户: 只有执行权限 = 1

该程序的权限为751。

64.以下哪个活动可以提高软件质量(D)。

A.需求分析 B.软件开发 C.软件测试 D.技术评审

- **需求分析**：软件开发的初始阶段，主要是为了确保需求的准确性和完整性。
- **软件开发**：编写代码的过程，虽然可以提高软件质量，但开发本身并不是一个专门的质量保证活动。
- **软件测试**：通过执行测试用例来发现和修复缺陷的过程。测试主要是发现和报告问题，而不是预防问题。
- **技术评审**：一种正式的技术审查过程，通过同行评审、代码审查、设计评审等方式，及早发现和纠正潜在的问题。技术评审可以在软件开发的各个阶段进行，包括需求分析、设计、编码和测试。它不仅有助于发现和修复缺陷，还可以提高团队的沟通和协作，从而提高软件的整体质量。

65.以下关于甘特图的叙述中，不正确的是(C)。

- A.一种进度管理的工具
- B.易于看出每个子任务的持续时间
- C.易于看出目前项目的实际进度情况
- D.易于看出子任务之间的衔接关系

甘特图是一种进度管理工具，易于看出每个子任务的持续时间、子任务之间的衔接关系。但是难以直接显示项目的实际进度情况，需要额外的信息或更新。

66.软件文档在软件生存期中起着重要的作用，其作用不包括(A)。

- A.提高软件运行效率
- B.作为开发过程的阶段工作成果和结束标记
- C.提高开发过程的能见度
- D.提高开发效率

软件运行效率通常取决于代码的质量、算法的选择和系统的设计等因素。

67.某个软件，不符合公司的战略决策，属于(A)风险。

- A.商业风险
- B.项目风险
- C.开发风险
- D.人员风险

- **商业风险**：指由于市场、竞争、财务、战略决策等因素导致的不确定性，可能会影响公司的业务目标和战略方向。
- **项目风险**：指在项目实施过程中可能出现的各种不确定因素，如进度延误、成本超支、资源不足等。这些风险主要集中在项目管理层面。
- **开发风险**：指在软件开发过程中可能出现的技术难题、设计缺陷、代码质量问题等。这些风险主要集中在技术实现层面。
- **人员风险**：指由于团队成员的能力、经验、离职等因素导致的风险。这些风险主要集中在人力资源管理层面。